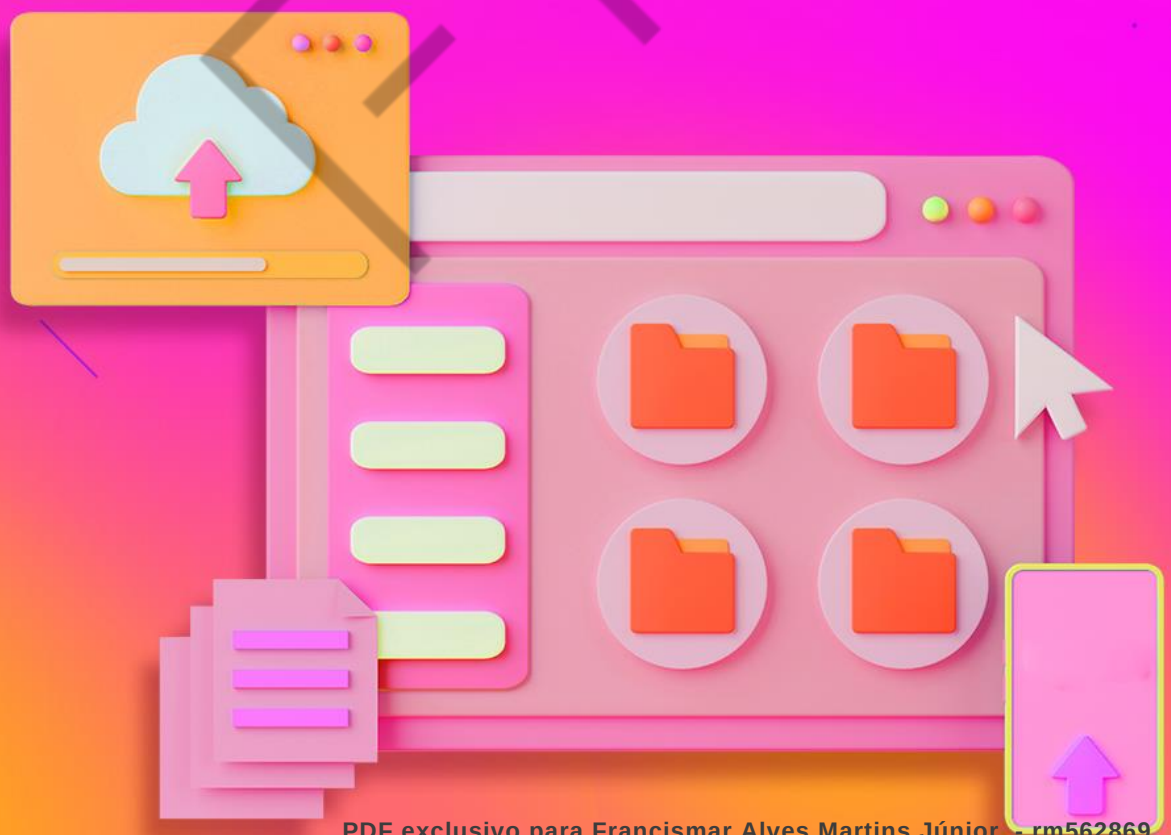
FIAP ON

BIG DATA

NOSQL COM MONGODB



5

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 – Resultado da execução do comando `createCollection`.....9
Figura 2 – Exemplo de uso dos comandos `show collections` e `createCollection`.....10
Figura 3 – Exibindo o conteúdo da constante `db`.....12
Figura 4 – Exemplo de uso dos comandos `use`.....14
Figura 5 – Resultado da operação de inclusão na coleção **emp**.....17
Figura 6 – Resultado da operação de inclusão com *array* na coleção **emp**.....18
Figura 7 – Resultado da operação de inclusão com a junção de outro documento na coleção **emp**.....20
Figura 8 – Resultado da operação de inclusão de um *array* de documentos na coleção **emp**.....21
Figura 9 – Resultado parcial da consulta aos dados da coleção **emp**.....24
Figura 10 – Resultado de uso do método `findOne()` em consulta aos dados da coleção **emp**.....25
Figura 11 – Resultado de uso do método `getTimestamp()`.....26
Figura 12 – Resultado da busca por igualdade e do método `pretty()`.....28
Figura 13 – MongoDB Download Center.....41
Figura 14 – Instalador do MongoDB Community Edition.....42
Figura 15 – Primeira tela do Instalador do MongoDB Community Edition.....42
Figura 16 – Termos e condições do Instalador do MongoDB Community Edition.....43
Figura 17 – Configurações do Instalador do MongoDB Community Edition.....43
Figura 18 – Passos finais do Instalador do MongoDB Community Edition.....43
Figura 19 – Tela final do Instalador do MongoDB Community Edition.....44

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE QUADROS

Quadro 1 – Comparação entre os termos usados pelo banco de dados relacional Oracle e o banco de dados orientado a documentos MongoDB. 7
Quadro 2 – Quadro com as principais funções de ajuda do banco de dados MongoDB 13
Quadro 3 – Principais usos do comando `show` do banco de dados MongoDB.....13
Quadro 4 – Símbolos e operadores usados em conjunto com os documentos MongoDB 14
Quadro 5 – Dados a serem incluídos na coleção grade.....22
Quadro 6 – Quadro com os principais operadores de comparação 30
Quadro 7 – Quadro de equivalência de operadores SQL versus MongoDB 30
Quadro 8 – Quadro com os principais operadores lógicos.....32
Quadro 9 – Quadro de equivalência de operadores lógicos SQL versus MongoDB .32
Quadro 10 – Quadro com os principais operadores de *arrays* 34
Quadro 11 – Quadro de equivalência de projeção SQL versus MongoDB 35
Quadro 12 – Quadro de equivalência de expressões regulares entre SQL versus MongoDB 37
Quadro 13 – Quadro de equivalência de operadores de atualização entre SQL versus MongoDB.....39
Quadro 14 – Quadro de equivalência de operações de remoção entre SQL versus MongoDB 40

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Sintaxe do comando createCollection....8
Código-fonte 2 – Exemplo do uso do comando createCollection....8
Código-fonte 3 – Exemplo do uso do comando `show collections`....9
Código-fonte 4 – Exemplo de documentos com dois campos....10
Código-fonte 5 – Sintaxe do comando `use`....14
Código-fonte 6 – Estrutura básica de um comando no banco de dados MongoDB ..15
Código-fonte 7 – Sintaxe do comando `insert`....16
Código-fonte 8 – Exemplo de inclusão de dados na coleção **emp**....17
Código-fonte 9 – Exemplo de inclusão de dados com *array* na coleção **emp**....18
Código-fonte 10 – Exemplo de inclusão de dados com a junção de outro documento na coleção **emp**....19
Código-fonte 11 – Exemplo de inclusão de dados com um `array` de documentos na coleção **emp**....21
Código-fonte 12 – Exemplo de criação de variável com oito documentos....23
Código-fonte 13 – Exemplo de inclusão de dados através de variável....23
Código-fonte 14 – Sintaxe do método `find()`....24
Código-fonte 15 – Exemplo de consulta aos documentos na coleção **emp**....24
Código-fonte 16 – Sintaxe do método `findOne()`....25
Código-fonte 17 – Exemplo de consulta ao primeiro documento na coleção **emp**....25
Código-fonte 18 – Exemplo de uso do método `getTimestamp()`....26
Código-fonte 19 – Exemplo de consulta por igualdade....27
Código-fonte 20 – Exemplo de consulta por igualdade com saída formatada pelo método `pretty()`....27
Código-fonte 21 – Exemplo de uso do método `sort()`....28
Código-fonte 22 – Exemplo de uso do operador `$lt`....29
Código-fonte 23 – Exemplo de uso do operador `$lte`....29
Código-fonte 24 – Exemplo de uso do operador `$lt`....29
Código-fonte 25 – Exemplo de uso do operador `$gte`....30
Código-fonte 26 – Exemplo de uso do operador `$or`....31
Código-fonte 27 – Exemplo de uso do operador `$and`....31
Código-fonte 28 – Exemplo de uso do operador `$nor`....31
Código-fonte 29 – Exemplo de uso do operador `$ne`....31
Código-fonte 30 – Exemplo de uso do operador `$not`....32
Código-fonte 31 – Exemplo de uso do operador `$in`....33
Código-fonte 32 – Exemplo de uso do operador `$nin`....33
Código-fonte 33 – Exemplo de uso do operador `$all`....33
Código-fonte 34 – Exemplo de uso do operador `$exists`....33
Código-fonte 35 – Exemplo de projeção....34
Código-fonte 36 – Sintaxe do método `find()`....35
Código-fonte 37 – Exemplo de expressão regular....36
Código-fonte 38 – Exemplo de expressão regular....36
Código-fonte 39 – Exemplo de expressão regular....36
Código-fonte 40 – Exemplo de expressão regular....36
Código-fonte 41 – Exemplo de expressão regular....36
Código-fonte 42 – Sintaxe do método `update()`....37

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Código-fonte 43 – Exemplo de alteração completa de um documento 38
Código-fonte 44 – Exemplo de uso do operador `$inc` 38
Código-fonte 45 – Exemplo de uso do operador `$set` 38
Código-fonte 46 – Exemplo de uso do operador `$rename` 39
Código-fonte 47 – Exemplo de uso do operador `$rename` 39
Código-fonte 48 – Sintaxe do método `remove()` 39
Código-fonte 49 – Exemplo de remoção documentos de uma coleção 40

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

NOSQL COM MONGODB.....7
1 INTRODUÇÃO 7
2 TERMINOLOGIA.....7
2.1 Coleção 8
2.2 Documento 10
3 TIPOS DE DADOS.....11
4 HELP 13
5 CRIANDO UM NOVO BANCO DE DADOS 14
6 OPERAÇÕES CRUD 15
7 INCLUINDO DADOS.....16
7.1 Inclusão de documentos por meio de variáveis.....22
8 CONSULTANDO DADOS 24
8.1 Atributo _id 26
8.2 Pesquisa por igualdade.....27
8.3 Classificando a saída dos dados.....28
8.4 Operadores de comparação.....29
8.5 Operadores lógicos 30
8.6 Operadores para tratamento *arrays* 33
8.7 Projeção 34
8.8 Expressões regulares.....35
9 ALTERANDO DADOS.....37
10 ELIMINANDO DADOS 39
11 INSTALAÇÃO MONGODB.....40
11.1 Download do instalador. 41
11.2 Execute o instalador do MongoDB Community Edition 41
11.3 Siga os passos do instalador do MongoDB Community Edition.....42
14 ESTUDO DE CASO 44
14.1 O problema.....44
14.2 Por que MongoDB?.....45
14.3 Modelo de documento fácil de implementar.....46
14.4 Tempo de resposta de milissegundos.....46
14.5 Altamente escalável 46
14.6 Alta disponibilidade 46
14.7 Resultados 47
REFERÊNCIAS.....47

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

NOSQL COM MONGODB

1 INTRODUÇÃO

O nome MongoDB deriva da palavra inglesa *humongous*, termo que pode ser traduzido como “gigantesco”. É um banco orientado a documentos e não possui conceito de tabelas, esquemas, transações, *joins*, e chaves estrangeiras (MONGODB, 2022).

Desenvolvido em C++, o MongoDB tem como filosofia facilitar o desenvolvimento de aplicações. É desprovido de esquema (*schema free* ou *schemaless*) e seus dados são armazenados no formato BSON (*Binary* JSON). Um único servidor MongoDB pode armazenar vários bancos.

2 TERMINOLOGIA

Para aqueles que conhecem bancos de dados relacionais, o quadro abaixo apresenta uma comparação entre os termos usados em um banco de dados relacional Oracle e um banco de dados orientado a documentos MongoDB.

Oracle (RDBMS)	MongoDB (documento)
Database	Database
Instância de Banco de Dados	Instância MongoDB
Esquema	Banco de Dados
Tabela, *View*	Coleção
Tupla	Documento (JSON, BSON)
Coluna	Campo ou *Field*
ROWID / Primary Key	_id
Junção	DBRef (Embedded Document)
Foreign Key	Reference
Partição	Shard
Select	Método *find*
Insert	Método *insert*
Update	Método *update*
Delete	Método *remove*

Quadro 1 – Comparação entre os termos usados pelo banco de dados relacional Oracle e o banco de dados orientado a documentos MongoDB.
Fonte: MongoDB (2022).

Entre esses termos daremos destaque a dois deles:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.1 Coleção

Uma coleção ou, *collection*, é onde os dados do MongoDB são armazenados. Uma coleção pode ser comparada com as tabelas de um banco de dados relacional, mas sua estrutura é muito menos rígida do que em um RDBMS. Uma coleção é um conjunto de documentos MongoDB. No próximo tópico falaremos sobre os documentos MongoDB (MONGODB, 2022).

Uma coleção pertence a um único banco de dados MongoDB e não possuem um esquema. Cada documento em uma coleção pode ter campos diferentes. Por uma questão de praticidade, procuramos criar coleções com documentos semelhantes (MONGODB, 2022).

As coleções podem ser criadas quando formos salvar um documento que a referencie ou através do comando `createCollection` (MONGODB, 2022).

```
db.createCollection(name, options)
```

Código-fonte 1 – Sintaxe do comando createCollection
Fonte: MongoDB (2022).

Onde:

Name – indica o nome da coleção a ser criada.

Options – podem assumir os seguintes valores:

- capped – coleção ordenada, circular, de tamanho fixo. Default “false”.
- autoIndexID – cria índice nos campos _id. Default “false”.
- size – define o tamanho, em bytes, de uma coleção capped.
- max – define o número máximo de documentos de uma coleção capped.

O comando abaixo cria uma coleção denominada **biblioteca**.

```
db.createCollection(“biblioteca”)
```

Código-fonte 2 – Exemplo do uso do comando createCollection
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A figura abaixo mostra o resultado da execução do comando:

```
> db.createCollection("biblioteca")
{ "ok" : 1 }
```

Figura 1 – Resultado da execução do comando `createCollection`
Fonte: Elaborado pelo autor (2022)

Podemos ver que a execução foi bem sucedida porque o comando retornou a chave valor `{ "ok" : 1 }`.

Os nomes de coleções devem obedecer aos seguintes critérios:

- O nome de uma coleção deve começar por uma letra ou sublinhado ("_").
- Pode conter números.
- Não pode utilizar o caractere cifrão ("$").
- Não pode exceder 128 caracteres.

É interessante notar que é possível criar grupos nomeados usando um ponto (“.”). Esses grupos são nomeados de *namespace collection*. Podemos, por exemplo, criar uma coleção para os livros de arte e outra para os romances:

- livros.romance
- livros.arte

O comando `show collections;` pode ser usado para exibir todas as coleções do banco de dados corrente.

```
show collections;
```

Código-fonte 3 – Exemplo do uso do comando `show collections`
Fonte: MongoDB (2022)

A imagem abaixo mostra o uso do comando `show collection;`, seguido da criação de uma coleção e a reexecução do comando `show collection;`.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
> show collections;
> db.createCollection("biblioteca");
{ "ok" : 1 }
> show collections;
biblioteca
```

Figura 2 – Exemplo de uso dos comandos `show collections` e `createCollection`.
Fonte: Elaborado pelo autor (2022)

Na figura 2 utilizamos o comando `show collections` como não havia nenhuma coleção criada, nenhum valor foi retornado, em seguida criamos a coleção biblioteca. Podemos ver que a execução foi bem sucedida porque o comando retornou a chave valor `{ "ok" : 1 }`. Em seguida, reexecutamos o comando `show collections`, desta vez o comando retornou o nome da coleção que acabamos de criar, biblioteca.

2.2 Documento

Em um banco MongoDB, um documento ou, *document*, é o equivalente a uma linha, ou tupla, em uma tabela de um banco relacional. O documento é representado por um conjunto de pares de chave-valor (MONGODB, 2022).

O exemplo abaixo mostra um documento com um nome e uma idade.

```
{
    Nome: 'Rita',
    Idade: 26
}
```

Código-fonte 4 – Exemplo de documentos com dois campos
Fonte: Elaborado pelo autor (2022)

Note que o documento é aberto com uma chave e é encerrado por outra chave, tudo que está entre as duas chaves faz parte deste documento. Existem dois pares de chave-valor, observe que cada par é separado por uma vírgula. O primeiro campo tem o nome de **Nome** e tem o valor '**Rita**', perceba que entre a chave e o valor existe o símbolo de dois pontos. O segundo campo tem o nome de **Idade** e tem o valor **26**.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Documentos na mesma coleção não precisam ter a mesma estrutura ou conjunto de campos. Documentos de uma mesma coleção podem conter diferentes tipos de dados (MONGODB, 2022).

3 TIPOS DE DADOS

O MongoDB possui suporte aos principais tipos de dados primitivos utilizados em qualquer banco de dados (MONGODB, 2022).

Abaixo, uma lista com os principais tipos de dados:

- String: Este é o tipo de dados mais comumente usado para armazenar dados. String no MongoDB deve ser um UTF-8 válido.
- Integer: Este tipo é usado para armazenar um valor numérico. Integer pode ser 32 bits ou 64 bits dependendo do seu servidor.
- Boolean: Este tipo é usado para armazenar um valor booleano (*true* ou *false*).
- Double: Este tipo é usado para armazenar valores de ponto flutuante.
- Min/Max keys: Este tipo é usado para comparar um valor contra os elementos mais baixos e mais altos de um BSON.
- Arrays: Este tipo é usado para armazenar *arrays*, listas ou múltiplos valores dentro de uma chave (*key*).
- Timestamp: ctimestamp. Isto pode ser útil para a gravação de quando um documento foi modificado ou acrescentado.
- Object: Este tipo de dado é usado para incorporar documentos.
- Null: Este tipo é usado para armazenar um valor nulo (*null*).
- Symbol: Este tipo de dados é usado de forma idêntica ao *String*, porém ele é geralmente reservado para linguagens que usam um tipo de símbolo específico.
- Date: Este tipo de dados é utilizado para armazenar a data ou a hora atual no formato de UNIX. Você pode especificar o seu próprio *date_time* através da criação do objeto Date e passando o dia, mês e ano para ele.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Object ID: Este tipo de dados é usado para armazenar os identificadores (ID) dos documentos.
- Binary data: Este tipo de dados é usado para armazenar um dado binário.
- Code: Este tipo de dados é usado para armazenar código javascript dentro do documento.
- Regular expression: Este tipo de dados é usado para armazenar expressões regulares.

Dentre os diversos tipos de dados os mais utilizados são os:

- Numéricos
- String (use ‘’ ou “” para identificá-los)
- Date
- Boolean (*true* ou *false*)
- Array

Como você já deve ter percebido, o banco de dados MongoDB pode conter uma ou mais coleções; uma coleção pode conter diferentes tipos de documentos e um documento pode conter um conjunto de chave-valor, *arrays* e documentos aninhados.

O nome das variáveis `shell` no MongoDB devem iniciar por uma letra minúscula [a-z], as letras seguintes podem ser minúsculas, maiúsculas ou números [a-zA-Z0-9] (MONGODB, 2022).

É possível visualizar o conteúdo de uma variável simplesmente digitando o nome da variável no terminal (MONGODB, 2022).

A imagem abaixo mostra um exemplo de uso.

```
connecting to: test
> db
test
```

Figura 3 – Exibindo o conteúdo da constante `db`
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4 HELP

O MongoDB possui vários comandos que fornecem ajuda ao desenvolvedor. O quadro abaixo mostra alguns desses comandos:

Comando	Função	Exemplo de Uso
help;	Exibe esta lista de funções.	help;
db.help();	Fornece uma lista das funções de ajuda para um determinado método do banco de dados.	db.help('insert');
db.<coleção>.help();	Fornece uma lista dos métodos aplicáveis a uma coleção. <coleção> indica o nome de uma coleção a ser criada ou previamente existente.	db.biblioteca.help();

Quadro 2 – Quadro com as principais funções de ajuda do banco de dados MongoDB
Fonte: MongoDB (2022)

O comando `show` também nos ajuda muito, pois com ele podemos obter informações sobre quais coleções existem em nosso banco, quais são os bancos de dados existentes no servidor e quais são os usuários do banco de dados corrente. Abaixo, um quadro com os principais usos do comando `show`.

Comando	Função
show dbs;	Fornece uma lista de todos os bancos de dados do servidor.
show collections;	Fornece uma lista de todas as coleções do banco de dados corrente.
show users;	Fornece uma lista de todos os usuários do banco de dados corrente.
show roles;	Fornece uma lista de todas as `roles`, predefinidas ou definidas pelo usuário, do banco de dados corrente.
show profile;	Fornece uma lista das últimas cinco operações que levaram um milissegundo ou mais para serem executadas.
show databases;	Fornece uma lista de todos os bancos de dados disponíveis.

Quadro 3 – Principais usos do comando `show` do banco de dados MongoDB
Fonte: MongoDB (2022)

Também nos ajuda saber quais são os símbolos e operadores que usamos para trabalhar com os documentos. O quadro abaixo lista os principais símbolos e operadores:

Operador / Símbolo	Significado
( ) (parênteses)	Indica o uso de um método.
[ ] (colchetes)	Usado em *arrays.*
{ } (chaves)	Usado em documentos.
. (ponto)	Separador de métodos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

= (igual)	Atribui valor para variáveis.
, (vírgula)	Separa atributos. Não deve ser usada com números.
: (dois pontos)	Especifica o valor de um atributo.
‘ (aspas simples)	Indica que o valor de um atributo é texto.
“ (aspas duplas)	Indica que o valor de um atributo é texto.
; (ponto-e-vírgula)	Encerra um comando.

Quadro 4 – Símbolos e operadores usados em conjunto com os documentos MongoDB
Fonte: MongoDB (2022)

5 CRIANDO UM NOVO BANCO DE DADOS

Ao contrário dos bancos de dados relacionais, não existe um comando `create database`. O banco é fisicamente criado no momento em que uma coleção é criada (MONGODB, 2022).

Para mudarmos de um banco para outro, usamos o comando `use`.

```
use(<db>)
```

Código-fonte 5 – Sintaxe do comando `use`
Fonte: MongoDB (2022).

Onde:

db – indica o nome do banco de dados a ser usado.

A variável *shell* `db` é alterada para o nome do banco de dados que será usado. A imagem abaixo mostra o uso do comando `use`.

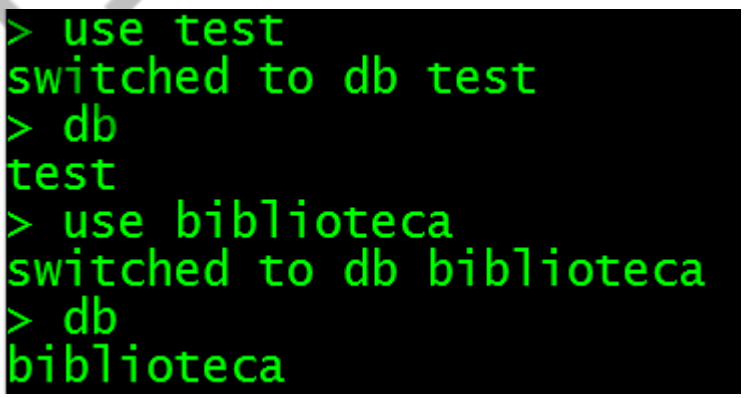


Figura 4 – Exemplo de uso dos comandos `use`
Fonte: Elaborado pelo autor (2022)

Na imagem, usamos o comando `use test` e, em seguida, exibimos o conteúdo da variável *shell* `db`, neste primeiro caso ela está com o valor **test**. A seguir,

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

usamos o comando `use biblioteca` e, novamente, exibimos o conteúdo da variável *shell* `db`, desta vez ela está com o valor **biblioteca**.

6 OPERAÇÕES CRUD

As quatro operações básicas em banco de dados são:

- Incluir dados (***c**reate*).
- Consultar dados (***r**ead*).
- Alterar dados (***u**pdade*).
- Apagar dados (***d**elete*).

O acrônimo das quatro palavras em inglês (*create*, *read*, *updade* e *delete*) forma a palavra CRUD. Foi-se convencionado que o termo CRUD indica essas quatro operações.

A estrutura básica de um comando no banco de dados MongoDB é:

```
database.coleção.função()
```

Código-fonte 6 – Estrutura básica de um comando no banco de dados MongoDB
Fonte: MongoDB (2022)

Onde:

database – indica o banco de dados. A variável `db` contêm o nome do banco de dados.

coleção – indica o nome da coleção.

função – indica a função que será aplicada a coleção.

Os comandos usados pelas operações CRUD, normalmente, começarão por `db` indicando que as operações serão realizadas no banco corrente (MONGODB, 2022).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

7 INCLUINDO DADOS

Adicionamos **documentos** a uma **coleção** através de uma operação de inclusão.

IMPORTANTE: Caso a coleção ainda não tenha sido criada, a operação de inclusão criará a coleção.

Existem várias formas de incluirmos dados em uma coleção. Neste primeiro momento, nos concentraremos na função `insert`.

```
db.coleção.insert(
   <documento ou array de documentos>,
   {
     writeConcern: <documento>,
     ordered: <boolean>
   }
)
```

Código-fonte 7 – Sintaxe do comando `insert`
Fonte: MongoDB (2022).

Onde:

Documento ou array de documentos – indica o documento ou array de documentos a serem inseridos em uma coleção.

writeConcern – define se a persistência dos dados será acknowledged (w) ou unacknowledged (j).

ordered – insere os documentos na sequência que estão no array. Se um erro ocorrer, os demais documentos do array não serão inseridos.

Vamos a um exemplo de uso. Imagine que você quer incluir dados de um funcionário em uma coleção denominada “emp”. Os dados do funcionário são:

Nome – Bianka

Idade – 22

Cargo – Desenvolvedor

Salario – 12.000,00

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Perceba que são quatro pares de chave-valor. A chave “Nome” tem o valor “Bianka”, a chave “Idade” tem o valor “22”, a chave “Cargo” tem o valor “Gerente” e a chave “Salario” tem o valor “12.000,00”. Vamos converter esses dados para uma estrutura de chave valor mais apropriada:

{

Nome: “Bianka”,

Idade: 22,

Cargo: ‘Desenvolvedor’,

Salario: 12000.00

}

Perceba que os campos que contêm texto por aspas simples ou aspas duplas, qualquer uma delas é aceita. Isso indica para o banco de dados que esse campo conterá um texto. Os campos numéricos não contêm aspas. As vírgulas são usadas para separar os pares de chave-valor. Os valores decimais são indicados por um ponto. Vamos usar a sintaxe para incluir os dados na coleção **emp**.

```
db.emp.insert({ Nome: "Bianka",
                Idade: 22,
                Cargo: 'Desenvolvedor',
                Salario: 12000.00
})
```

Código-fonte 8 – Exemplo de inclusão de dados na coleção **emp**
Fonte: Elaborado pelo autor (2022)

A imagem abaixo mostra o resultado da operação.

```
> db.emp.insert({ Nome: "Bianka", Idade: 22, Cargo: 'Desenvolvedor', Salario: 12000.00 })
WriteResult({ "nInserted" : 1 })
>
```

Figura 5 – Resultado da operação de inclusão na coleção **emp**
Fonte: Elaborado pelo autor (2022)

Note que o resultado da operação foi a mensagem de “`WriteResult({ “nInserted” : 1 })`. Isso indica que o comando foi bem sucedido e que um documento foi incluído na coleção **emp**.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Vamos a outro exemplo, desta vez os dados são os seguintes:

Nome – Neusa

Idade – 23

Cargo – Desenvolvedor

Salario – 12.830,26

Dependentes – Maria, Pietra

Note que, desta vez acrescentamos a chave “Dependentes”, que possui dois valores “Maria” e “Pietra”. Como essa chave possui dois valores, usaremos um *array* para representá-lo. Lembrando que campos do tipo texto podem ser indicados usando aspas simples ou aspas duplas.

```
db.emp.insert(
              { Nome: "Neusa",
                Idade: 23,
                Cargo: 'Desenvolvedor',
                Salario: 12830.26,
                Dependentes: ["Maria", 'Pietra']
}
)
```

Código-fonte 9 – Exemplo de inclusão de dados com *array* na coleção **emp**
Fonte: Elaborado pelo autor (2022)

A imagem abaixo mostra o resultado da operação.

```
> db.emp.insert({ Nome: "Neusa",
...                Idade: 23,
...                Cargo: 'Desenvolvedor',
...                Salario: 12830.26,
...                Dependentes: ["Maria", 'Pietra']
... })
WriteResult({ "nInserted" : 1 })
>
```

Figura 6 – Resultado da operação de inclusão com *array* na coleção **emp**
Fonte: Elaborado pelo autor (2022)

Vamos a mais um exemplo um pouco mais complexo. Desta vez, vamos incluir informações sobre o departamento onde o empregado trabalha.

Os dados do funcionário são:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Nome – Thais

Idade – 19

Cargo – Analista

Salario – 14.530,77

Dependentes – Angelina, Dora, Hugo

Os dados do departamento são:

Nome – Pesquisa

Local – São Paulo

Vamos tratar os dados do departamento como um documento dentro do documento.

```
db.emp.insert(
              { Nome: "Thais",
                Idade: 19,
                Cargo: "Analista",
                Salario: 14530.77,
                Dependentes:["Angela", "Dora", "Hugo"],
                Departamento: { Nome: "Pesquisa",
                                Local: "São Paulo"
                              }

}
)
```

Código-fonte 10 – Exemplo de inclusão de dados com a junção de outro documento na coleção **emp**
Fonte: Elaborado pelo autor (2022)

Note que, ao contrário dos bancos de dados relacionais, procuramos evitar a junção de duas coleções. Sempre que possível já efetuamos a operação de junção durante a inclusão dos dados. A imagem abaixo mostra o resultado da operação.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
> db.emp.insert({ Nome: "Thais",
...                Idade: 19,
...                Cargo: "Analista",
...                Salario: 14530.77,
...                Dependentes:["Angela", "Dora", "Hugo"],
...                Departamento: { Nome: "Pesquisa",
...                                Local: "São Paulo"
...                              }
...
... })
WriteResult({ "nInserted" : 1 })
>
```

Figura 7 – Resultado da operação de inclusão com a junção de outro documento na coleção **emp**
Fonte: Elaborado pelo autor (2022)

Podemos ter vários documentos dentro do mesmo documento. Também podemos ter vetores de documentos. Vamos incluir no documento do funcionário as informações sobre as suas promoções dentro da empresa.

Os dados do funcionário são:

Nome – Fátima

Idade – 29

Cargo – Analista

Salario – 12.345,67

Dependentes – Gohan

Os dados do departamento são:

Nome – Vendas

Local – Campinas

O histórico de promoções é:

Ano – 2001

Cargo – "Estagiário"

Valor – 180,00

Ano – 2002

Cargo - "Desenvolvedor "

Valor – 1.700,00

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Vamos tratar os dados das promoções como um *array* de documentos dentro do documento.

```
db.emp.insert(
              { Nome: "Fátima",
                Idade: 29,
                Cargo: "Analista",
                Salario: 12345.67,
                Dependentes:["Gohan"],
                Departamento: { Nome: "Vendas",
                                Local: "Campinas"
                              },
                Promoções: [{Ano: 2001,
                             Cargo: "Estagiário",
                             Valor: 180.00},
                            {Ano: 2002,
                             Cargo: "Analista",
                             Valor: 1700.00}]
  }
  )
```

Código-fonte 11 – Exemplo de inclusão de dados com um `array` de documentos na coleção **emp**
Fonte: Elaborado pelo autor (2022)

Note que temos um *`array`* de documentos em promoções. A imagem abaixo mostra o resultado dessa inclusão.

```
> db.emp.insert({ Nome: "Fátima",
...               Idade: 29,
...               Cargo: "Analista",
...               Salario: 12345.67,
...               Dependentes:["Gohan"],
...               Departamento: { Nome: "Vendas",
...                               Local: "Campinas"
...                             },
...               Promoções: [{Ano: 2001,
...                            Cargo: "Estagiário",
...                            Valor: 180.00},
...                           {Ano: 2002,
...                            Cargo: "Analista",
...                            Valor: 1700.00}]
...    })
WriteResult({ "nInserted" : 1 })
>
```

Figura 8 – Resultado da operação de inclusão de um *array* de documentos na coleção **emp**
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

7.1 Inclusão de documentos por meio de variáveis

O uso de variáveis permite que incluamos vários documentos em uma única operação.

Imagine que queira incluir os seguintes dados em uma coleção denominada grade:

Nome	Autor	Professor	Novo	Cód	Mês	ISBN	Valor
Estruturas de dados	**Rossetti**		**FALSE**			**12345678**	**171,13**
Engenharia de software		**Rita**		**FES**	**2**		
Inglês instrumental	**Ana**		**FALSE**			**45678901**	**122,99**
Interfaces com usuário		**André**		**IU**	**2**		
Programação orientada a objeto	**Gatti**		**TRUE**			**23456789**	**161,16**
Modelagem orientada a objeto		**Rodrigo**		**MOO1**	**2**		
Banco de dados	**Angélica**		**TRUE**			**34567890**	**133,29**
Gestão empresarial	**Silva**	**Silva**	**TRUE**	**GE**	**2**	**56789012**	**156,55**

Quadro 5 – Dados a serem incluídos na coleção grade
Fonte: Elaborado pelo autor (2022)

É possível criar uma variável usando a instrução **var** que receberá esses documentos.

```
var dados = [
{Nome: 'Estruturas de dados',
 Autor: 'Rossetti',
 Novo: false,
 ISBN: 12345678,
 Valor: 171.13},

{Nome: 'Engenharia de software',
 Professor: 'Rita',
 Cod: 'FES', Mes: 2},

{Nome: 'Inglês instrumental',
 Autor: 'Ana',
 Novo: false,
 ISBN: 45678901,
 Valor: 122.99},
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
{Nome: 'Interfaces com usuário',
 Professor: 'André',
 Cod: 'IU',
 Mes: 2},

{Nome: 'Programação orientada a objeto',
 Autor: 'Gatti',
 Novo: true,
 ISBN: 23456789,
 Valor: 161.16},

{Nome: 'Modelagem orientada a objeto',
 Professor: ‘Rodrigo',
 Cod: 'MOO1',
 Mes: 2},

{Nome: 'Banco de dados',
 Autor: 'Angélica',
 Novo: true,
 ISBN: 34567890,
 Valor: 133.29},

{Nome: 'Gestão empresarial',
 Autor: ‘Silva',
 Professor: ‘Silva',
 Novo: true,
 Cod: 'GE',
 Mes: 2,
 ISBN: 56789012, Valor: 156.55}
 ]
```

Código-fonte 12 – Exemplo de criação de variável com oito documentos
Fonte: Elaborado pelo autor (2022)

Essa variável pode ser usada junto com o método `insert()` para incluir os documentos na coleção grade.

```
db.grade.insert(dados);
```

Código-fonte 13 – Exemplo de inclusão de dados através de variável
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

8 CONSULTANDO DADOS

Os documentos que foram incluídos nas coleções podem ser consultados usando o método `find()`. Esse método retorna um *array* com os objetos da coleção, mesmo que a coleção tenha apenas um objeto (MONGODB, 2022).

```
db.coleção.find(consulta, projeção)
```

Código-fonte 14 – Sintaxe do método `find()`
Fonte: MongoDB (2022).

Onde:

Coleção – Indica o nome da coleção.

Consulta – Indica quais serão os filtros aplicados na pesquisa. Para retornar todos os documentos de uma coleção basta omitir este parâmetro ou informar um documento vazio `({})`. Este parâmetro é opcional.

Projeção – Indica quais são os campos específicos do documento que serão retornados pela pesquisa. Se omitido, todos os campos do documento serão retornados. Este parâmetro é opcional.

A consulta básica aos dados de uma coleção MongoDB é bem simples. Vejamos um exemplo de uso:

```
db.emp.find();
```

Código-fonte 15 – Exemplo de consulta aos documentos na coleção **emp**
Fonte: Elaborado pelo autor (2022)

A imagem abaixo mostra o resultado parcial da execução do comando.

```
> db.emp.find();
{ "_id" : ObjectId("5b3bee4ba8a46b85015e70ed"), "Nome" : "Bianka", "Idade" : 22, "Cargo" : "Desenvolvedor",
{ "_id" : ObjectId("5b3bfa3da8a46b85015e70ee"), "Nome" : "Neusa", "Idade" : 23, "Cargo" : "Desenvolvedor",
{ "_id" : ObjectId("5b3c09f2a8a46b85015e70ef"), "Nome" : "Thais", "Idade" : 19, "Cargo" : "Analista", "Sala
{ "_id" : ObjectId("5b3c1673a8a46b85015e70f0"), "Nome" : "Fátima", "Idade" : 29, "Cargo" : "Analista", "Sal
>
```

Figura 9 – Resultado parcial da consulta aos dados da coleção **emp**
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O método `find()` retornou uma lista com todos os documentos da coleção. Podemos usar o método `findOne()` para retornar o primeiro objeto encontrado na coleção que atenda aos critérios de busca da consulta (MONGODB, 2022).

```
db.coleção.findOne(consulta, projeção)
```

Código-fonte 16 – Sintaxe do método `findOne()`
Fonte: MongoDB (2022)

Onde:

Coleção – Indica o nome da coleção.

Consulta – Indica quais serão os filtros aplicados na pesquisa. Para retornar todos os documentos de uma coleção basta omitir este parâmetro ou informar um documento vazio `({})`. Este parâmetro é opcional.

Projeção – Indica quais são os campos específicos do documento que serão retornados pela pesquisa. Se omitido, todos os campos do documento serão retornados. Este parâmetro é opcional.

A consulta básica usando o `findOne()` é bem simples. Vejamos um exemplo de uso:

```
db.emp.findOne();
```

Código-fonte 17 – Exemplo de consulta ao primeiro documento na coleção **emp**
Fonte: Elaborado pelo autor (2022)

A imagem abaixo mostra o resultado da execução do comando.

```
> db.emp.findOne();
{
        "_id" : ObjectId("5b3bee4ba8a46b85015e70ed"),
        "Nome" : "Bianka",
        "Idade" : 22,
        "Cargo" : "Desenvolvedor",
        "Salario" : 12000
}
>
```

Figura 10 – Resultado de uso do método `findOne()` em consulta aos dados da coleção **emp**
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Note que apenas os dados do primeiro objeto da coleção foram exibidos. Perceba que há um atributo novo em nosso documento, **_id**.

8.1 Atributo _id

O atributo **_id** tem o comportamento similar a uma chave primária em bancos de dados relacional. Cada documento possui seu próprio **_id** e o seu número é único.

Um índice único (*unique index*) é criado para o atributo **_id** quando a coleção é criada. Ele sempre é o primeiro campo de um documento, se não for informado durante a inclusão de dados um tipo de dados **ObjectId** do **BSON** com 12 *bytes* será atribuído a ele. Se o valor do **_id** for informado e ele não for o primeiro campo, o servidor desloca o atributo para a primeira posição.

Normalmente a estrutura do **_id** segue a seguinte composição:

- Os quatro primeiros bytes indicam o *timestamp* da criação do objeto.
- 3 bytes identificam a máquina.
- 2 bytes mostram o identificador (id) do processo.
- 3 bytes contador, iniciado com um valor aleatório.

Ordenar os documentos pelo **_id** é o equivalente a ordená-los pela data de criação. Podemos usar o método `getTimestamp()` para verificarmos a data de criação de um objeto.

Exemplo de uso do método `getTimestamp()`.

```
ObjectId("5b3bee4ba8a46b85015e70ed").getTimestamp();
```

Código-fonte 18 – Exemplo de uso do método `getTimestamp()`
Fonte: Elaborado pelo autor (2022)

A imagem abaixo exibe o resultado da execução do comando.

```
> ObjectId("5b3bee4ba8a46b85015e70ed").getTimestamp()
ISODate("2018-07-03T21:44:43Z")
>
```

Figura 11 – Resultado de uso do método `getTimestamp()`
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Note que o objeto foi criado no dia 03 de julho de 2018 às 21:44:43 do horário local.

8.2 Pesquisa por igualdade

Uma consulta vazia ({}) retorna todos os documentos de uma coleção, não especificar uma consulta é equivalente a uma consulta vazia. Isso quer dizer que o comando `db.emp.find()` é equivalente a `db.emp.find({})`.

Listar todos os dados de uma coleção é muito útil, mas, algumas vezes, temos a necessidade de fazer consultas por igualdade. Para isso, basta usarmos o método `find()` com uma chave-valor. Por exemplo:

```
db.emp.find({Idade: 19});
```

Código-fonte 19 – Exemplo de consulta por igualdade
Fonte: Elaborado pelo autor (2022)

Nesse exemplo serão listados todos os documentos cujo atributo **Idade** seja igual a 19. O método `find()` pode encadear vários métodos para melhorar a busca de documentos. Por exemplo, podemos encadear o método `pretty()` parar formatar a saída de nossa consulta e deixá-la mais legível. O exemplo abaixo ilustra isso.

```
db.emp.find({Idade: 19}).pretty();
```

Código-fonte 20 – Exemplo de consulta por igualdade com saída formatada pelo método `pretty()`
Fonte: Elaborado pelo autor (2022)

A imagem abaixo ilustra a execução do comando.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
> db.emp.find({Idade: 19}).pretty();
{
        "_id" : ObjectId("5b3c09f2a8a46b85015e70ef"),
        "Nome" : "Thais",
        "Idade" : 19,
        "Cargo" : "Analista",
        "Salario" : 14530.77,
        "Dependentes" : [
                "Angela",
                "Dora",
                "Hugo"
        ],
        "Departamento" : {
                "Nome" : "Pesquisa",
                "Local" : "São Paulo"
        }
}
```

Figura 12 – Resultado da busca por igualdade e do método `pretty()`
Fonte: Elaborado pelo autor (2022)

8.3 Classificando a saída dos dados

Vários outros métodos podem ser concatenados ao método `find()`, o método `sort()`, por exemplo, classifica a saída em ordem crescente ou decrescente.

- O valor 1 no parâmetro indica que os dados serão classificados em ordem crescente.
- O valor -1 no parâmetro indica que os dados serão classificados em ordem decrescente.

Vejamos alguns exemplos de utilização:

```
db.emp.find().sort({Nome:1});
db.emp.find().sort({Idade:-1});
db.emp.find().sort({Cargo:1, Idade:-1});
```

Código-fonte 21 – Exemplo de uso do método `sort()`
Fonte: Elaborado pelo autor (2022)

No exemplo, o primeiro comando retorna todos os documentos da coleção **emp** classificados pelo nome em ordem crescente; o segundo comando retorna todos os

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

documentos da coleção classificados pela idade em ordem decrescente e o último comando retorna todos os documentos classificados pelo cargo em ordem crescente e pela idade em ordem decrescente.

8.4 Operadores de comparação

Os operadores de comparação retornam um valor booleano. Comparam as duas expressões informadas nos argumentos e retornam o valor *true* ou *false*.

- `$lt` (abreviação de *less then*) usado parar verificar se um valor é menor que outro. Retorna *true* se o primeiro valor for menor que o segundo.

```
db.emp.find({Idade:{$lt: 22}});
```

Código-fonte 22 – Exemplo de uso do operador `$lt`
Fonte: Elaborado pelo autor (2022)

- `$lte` (abreviação de *less then or igual*) usado para verificar se um valor é menor ou igual a outro. Retorna *true* se o primeiro valor for menor ou igual ao segundo.

```
db.emp.find({Idade:{$lt: 22}});
```

Código-fonte 23 – Exemplo de uso do operador `$lte`
Fonte: Elaborado pelo autor (2022)

- `$gt` (abreviação de *greater then*) usado parar verificar se um valor é maior que outro. Retorna *true* se o primeiro valor for maior que o segundo.

```
db.emp.find({Idade:{$lt: 22}});
```

Código-fonte 24 – Exemplo de uso do operador `$lt`
Fonte: Elaborado pelo autor (2022)

- `$gte` (abreviação de *greater then or equal*) usado parar verificar se um valor é maior ou igual a outro. Retorna *true* se o primeiro valor for maior ou igual ao segundo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
db.emp.find({Idade:{$lt: 22}});
```

Código-fonte 25 – Exemplo de uso do operador `$gte`
Fonte: Elaborado pelo autor (2022)

O quadro abaixo resume esses operadores e o exemplificam.

Operador	Significado	Exemplo de Uso
$lt	Menor que	db.emp.find({Idade:{$lt: 23}});
$lte	Menor que ou igual a	db.emp.find({Idade:{$lte: 23}});
$gt	Maior que	db.emp.find({Idade:{$gt: 23}});
$gte	Maior que ou igual a	db.emp.find({Idade:{$gte: 23}});

Quadro 6 – Quadro com os principais operadores de comparação
Fonte: Adaptado de MongoDB (2022)

O exemplo do operador `$lt`, exibido no quadro, retorna todos os documentos onde o campo **Idade** tenha o valor menor do que 23.

O exemplo do operador `$lte`, exibido no quadro, retorna todos os documentos onde o campo **Idade** tenha o valor menor ou igual a 23.

O exemplo do operador `$gt`, exibido no quadro, retorna todos os documentos onde o campo **Idade** tenha o valor maior do que 23.

O exemplo do operador `$gte`, exibido no quadro, retorna todos os documentos onde o campo **Idade** tenha o valor maior ou igual a 23.

O quadro abaixo compara o uso de comandos SQL dos bancos de dados relacional e o MongoDB.

Exemplo Em SQL	Exemplo em MongoDB
SELECT * FROM emp	db.emp.find()
SELECT * FROM emp WHERE idade = 33	db.emp.find({idade: 33})
SELECT * FROM emp WHERE idade > 33	db.emp.find({idade: {$gt: 33}})
SELECT * FROM emp WHERE idade >= 33	db.emp.find({idade: {$gte: 33}})
SELECT * FROM emp WHERE idade < 33	db.emp.find({idade: {$lt: 33}})
SELECT * FROM emp WHERE idade <= 33	db.emp.find({idade: {$lte: 33}})

Quadro 7 – Quadro de equivalência de operadores SQL versus MongoDB
Fonte: Adaptado de MongoDB (2022)

8.5 Operadores lógicos

Além dos operadores de comparação, podemos usar os operadores lógicos:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- `$or` retorna documentos pesquisados quando pelo menos uma das cláusulas de comparação forem verdadeiras.

```
db.emp.find({$or:[{Idade:{$gt:21}},{Salario:{$lt:12500}}]});
```

Código-fonte 26 – Exemplo de uso do operador `$or`
Fonte: Elaborado pelo autor (2022)

- `$and` retorna documentos pesquisados quando todas as cláusulas de comparação forem verdadeiras.

```
db.emp.find({$and:[{Idade:{$gt:21}},{Salario:{$lt:12500}}]});
```

ou

```
db.emp.find({Idade: {$gt: 21},Salario:{$lt: 12500}});
```

Código-fonte 27 – Exemplo de uso do operador `$and`
Fonte: Elaborado pelo autor (2022)

- `$nor` retorna documentos pesquisados que não satisfaçam as cláusulas de comparação.

```
db.emp.find({$nor:[{Idade:{$gt:22}},{Salario:{$gt:12500}}]});
```

Código-fonte 28 – Exemplo de uso do operador `$nor`
Fonte: Elaborado pelo autor (2022)

- `$ne` retorna documentos pesquisados cujos valores sejam diferentes do valor informado.

```
db.emp.find({Idade:{$ne:22}});
```

Código-fonte 29 – Exemplo de uso do operador `$ne`
Fonte: Elaborado pelo autor (2022)

- `$not` efetua uma operação NOT lógica.

```
db.emp.find({Idade:{$not:{$gt:21}}});
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Código-fonte 30 – Exemplo de uso do operador `$not`
Fonte: Elaborado pelo autor (2022)

O quadro abaixo resume esses operadores e os exemplificam.

Operador	Significado	Exemplo de Uso
$or	Ou lógico	db.emp.find({$or:[{Idade:{$lt:21}}, {Salario:{$gt:12500}}]});
$and	E lógico	db.emp.find({Idade: {$lt: 21},Salario:{$gt: 12500}});
$nor	NOR lógico	db.emp.find({$nor:[{Idade:{$lt:21}}, {Salario:{$gt:12500}}]});
$ne	Diferente	db.emp.find({Idade:{$ne:21}});

Quadro 8 – Quadro com os principais operadores lógicos
Fonte: Adaptado de MongoDB (2022)

O exemplo do operador `$or`, exibido no quadro, retorna todos os documentos onde o campo **Idade** tenha o valor menor do que 21 ou o campo **Salario** for maior que 12.500,00.

O exemplo do operador `$and`, exibido no quadro, retorna todos os documentos onde o campo **Idade** tenha o valor menor que 21 e o campo **Salario** for maior que 12.500,00.

O exemplo do operador `$nor`, exibido no quadro, retorna todos os documentos onde o campo **Idade** tenha o valor maior do que 21 e o campo **Salario** for menor que 12.500,00.

O quadro abaixo compara o uso de comandos SQL dos bancos de dados relacional e o MongoDB.

Exemplo Em SQL	Exemplo em MongoDB
SELECT * FROM emp WHERE Idade > 33 AND Idade < 40	db.emp.find({Idade: {$gt: 33, $lt: 40}})
SELECT * FROM emp WHERE Idade = 32 AND Nome = ‘Thais’	db.emp.find({Idade: 32, Nome: "Thais"})
SELECT * FROM emp WHERE Idade = 33 OR Nome = ‘Thais’	db.emp.find({$or:[{Idade:33}, {Nome: "Thais"}]})
SELECT * FROM emp WHERE Idade >= 32 AND (Nome = ‘Thais’ OR Cargo=‘Analista’)	db.emp.find(Idade: {$gte:32}, $or: [{Nome: "Thais"}, {Cargo: "Analista"}] })
SELECT * FROM emp WHERE Idade > 33 AND Idade < 40	db.emp.find({Idade: {$gt: 33, $lt: 40}})
SELECT * FROM emp WHERE Idade = 32 AND Nome = ‘Thais’	db.emp.find({Idade: 32, Nome: "Thais"})

Quadro 9 – Quadro de equivalência de operadores lógicos SQL versus MongoDB
Fonte: Adaptado de MongoDB (2022)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

8.6 Operadores para tratamento *arrays*

O MongoDB também fornece operadores para tratarmos *arrays*:

- `$in` retorna documentos onde o valor pesquisado esteja na lista de valores de um *array*.

```
db.emp.find({Dependentes:{$in:["Rosa", "Hugo"]}});
```

Código-fonte 31 – Exemplo de uso do operador `$in`
Fonte: Elaborado pelo autor (2022)

- `$nin` retorna documentos onde o valor pesquisado não esteja na lista de valores de um *array* ou o campo pesquisado não exista no documento.

```
db.emp.find({Dependentes:{$nin:["Rosa", "Rita"]}});
```

Código-fonte 32 – Exemplo de uso do operador `$nin`
Fonte: Elaborado pelo autor (2022)

- `$all` retorna documentos onde todos valores pesquisados estejam na lista de valores de um *array*.

```
db.emp.find({Dependentes:{$all:["Rita", "Ana"]}});
```

Código-fonte 33 – Exemplo de uso do operador `$all`
Fonte: Elaborado pelo autor (2022)

- `$exists` não específico para o uso em *arrays* mas pode ser usado para complementar os operadores `$in` e `$nin`. É usado para testar se um campo existe em um documento ou não. No exemplo usamos o método `count()` que efetua a contagem do número de elementos retornados.

```
db.emp.find({Dependentes:{$exists: true}}).count()
```

ou

```
db.emp.find({Dependentes:{$exists: false}}).count()
```

Código-fonte 34 – Exemplo de uso do operador `$exists`

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Fonte: Elaborado pelo autor (2022)

O quadro abaixo mostra esses operadores e os exemplificam.

Operador	Exemplo de Uso
$in	db.emp.find({Dependentes:{$in:["Dora", "Pietra"]}});
$nin	db.emp.find({Dependentes:{$nin:["Dora", "Pietra"]}});
$all	db.emp.find({Dependentes:{$all:["Dora", "Hugo"]}});
$exists	db.emp.find({Dependentes:{$exists:true, $nin:["Dora", "Pietra"]}});

Quadro 10 – Quadro com os principais operadores de *arrays*
Fonte: Adaptado de MongoDB (2022)

O exemplo do operador `$in`, exibido no quadro, retorna todos os documentos onde o campo **Dependentes** tenha o valor “Dora” ou o valor “Pietra”.

O exemplo do operador `$nin`, exibido no quadro, retorna todos os documentos onde o campo **Dependentes** não exista ou exista, mas não contenha o valor “Dora” ou o valor “Pietra”.

O exemplo do operador `$all`, exibido no quadro, retorna todos os documentos onde o campo **Dependentes** contenha tanto o valor “Dora” quanto o valor “Hugo”.

O exemplo do operador `$exists`, exibido no quadro, retorna todos os documentos onde o campo **Dependentes** exista e não contenha o valor “Dora” ou o valor “Pietra”.

8.7 Projeção

O uso do opcional de projeção permite indicar quais campos serão retornados pelo método `find()`.

```
db.emp.find({Idade:22},{_id:0, Idade:0, Cargo:0});
```

Código-fonte 35 – Exemplo de projeção
Fonte: Elaborado pelo autor (2022)

No exemplo, serão exibidos todos os campos dos funcionários com idade igual a 22 anos, exceto os campos **_id**, **Idade** e **Cargo**.

O quadro abaixo compara o uso de comandos SQL dos bancos de dados relacional e o MongoDB.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Exemplo Em SQL	Exemplo em MongoDB
SELECT * FROM emp	db.emp.find()
SELECT Nome, Salario FROM emp WHERE Idade = 22	db.emp.find({Idade:22},{_id:0, Idade:0, Cargo:0})
SELECT Nome, Salario FROM emp WHERE Idade = 22	db.emp.find({Idade:22},{Nome:1, Salario:1})

Quadro 11 – Quadro de equivalência de projeção SQL versus MongoDB
Fonte: Adaptado de MongoDB (2022)

8.8 Expressões regulares

Em MongoDB, expressões regulares permitem pesquisar por padrões em campos texto. Há duas formas para a sintaxe de expressões regulares (MONGODB, 2022).

```
{<campo>: { $regex: /padrão/, $options: '<opções>' } }
{<campo>: { $regex: 'padrão', $options: '<opções>' } }
{<campo>: { $regex: /padrão/<opções> } }

                    Ou

{ <campo>: /padrão/<opções> }
```

Código-fonte 36 – Sintaxe do método `find()`
Fonte: MongoDB (2022)

Onde:

Campo – Indica o campo onde será pesquisado o padrão.

Padrão – Indica o padrão a ser pesquisado.

Opções – Indica as opções a serem usadas com a expressão regular. Algumas das opções são:

- `i` – Indica *case insensitive*, isto é, o texto pode estar tanto em letras maiúsculas quanto em letras minúsculas.
- `m` – Usado em padrões que incluem âncoras, isto é, ^ para o início de um texto e $ para o final do texto.
- `x` – Usado para ignorar todos os espaços em branco no padrão.

Alguns exemplos:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
db.emp.find({Nome:{$regex:"^B"}});
```

Código-fonte 37 – Exemplo de expressão regular
Fonte: Elaborado pelo autor (2022)

No exemplo, estamos pesquisando por todos os nomes iniciados pela letra “B” maiúscula.

```
db.emp.find({Nome:{$regex:"^B", $options: 'i'}});
```

Código-fonte 38 – Exemplo de expressão regular
Fonte: Elaborado pelo autor (2022)

No exemplo, estamos pesquisando por todos os nomes iniciados pela letra “B”, independentemente de ter sido digitado em letra maiúscula ou letra minúscula.

```
db.emp.find({Nome:{$regex:"i"}});
```

Código-fonte 39 – Exemplo de expressão regular
Fonte: Elaborado pelo autor (2022)

No exemplo, estamos pesquisando por todos os nomes que possuem a letra “i”, minúscula, em qualquer parte do nome.

```
db.emp.find({Nome:{$regex:"i", $options: 'i'}});
```

Código-fonte 40 – Exemplo de expressão regular
Fonte: Elaborado pelo autor (2022)

No exemplo, estamos pesquisando por todos os nomes que possuem a letra “i”, independentemente de ter sido digitada em letra maiúscula ou letra minúscula, em qualquer parte do nome.

```
db.emp.find({Nome:{$regex:"s$"}});
```

Código-fonte 41 – Exemplo de expressão regular
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

No exemplo, estamos pesquisando por todos os nomes terminados pela letra “s”, minúscula.

O quadro abaixo compara o uso de comandos SQL dos bancos de dados relacional e o MongoDB.

Exemplo Em SQL	Exemplo em MongoDB
SELECT * FROM emp WHERE nome LIKE ‘%h%’	`db.emp.find({Nome:{$regex:"h"}});`

Quadro 12 – Quadro de equivalência de expressões regulares entre SQL versus MongoDB
Fonte: Adaptado de MongoDB (2022)

9 ALTERANDO DADOS

O método `update()` permite alterar um ou mais documentos em uma coleção. É possível alterar um ou mais campos de um documento ou, até mesmo, o documento inteiro. Por padrão, o método `update()` altera apenas um documento por vez (MONGODB, 2022). Sua sintaxe é:

```
db.coleção.update(consulta, atualização, opções)
```

Código-fonte 42 – Sintaxe do método `update()`
Fonte: MongoDB (2022)

Onde:

Coleção – Indica o nome da coleção.

Consulta – Indica quais serão os critérios para pesquisar o documento que será alterado.

Atualização – Indica a modificação que será feita.

Opções – Indica quais são as opções para o comando. Entre as opções estão:

- `upsert` – Opcional. Se definido como *true* cria um novo documento caso nenhum documento atenda os critérios de busca. O padrão é *false*.
- `multi` – Opcional. Se definido como *true* atualiza todos os documentos que atendam os critérios de busca. O padrão é *false*.

O método update() pode alterar um documento inteiro. O exemplo abaixo ilustra o caso.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
db.paises.find();

db.paises.insert({_id:504,Nome:"Mauritânia",Sobrenome:
"Nouakchott"});

db.paises.find();

db.paises.update({_id:504},{Pais:"Honduras",Capital:"Teg
ucigalpa"});
```

Código-fonte 43 – Exemplo de alteração completa de um documento
Fonte: Elaborado pelo autor (2022)

Neste caso, o documento inteiro foi substituído.

O método `update()` tem operadores para tratar os campos individualmente. Entre eles destacamos:

- `$inc` incrementa o valor de um campo pelo valor informado no operador.

```
db.emp.insert({Nome: "Ana", idade: 22});
db.emp.find({Nome: "Ana"});
db.emp.update({Nome: "Ana"}, {$inc: { idade: 5}});
db.emp.find({Nome: "Ana"});
```

Código-fonte 44 – Exemplo de uso do operador `$inc`
Fonte: Elaborado pelo autor (2022)

- `$set` modifica o conteúdo de um campo específico. Caso o campo não exista, um novo campo será acrescido ao documento.

```
db.emp.update({Nome: "Ana"}, {$set: { idade: 22}});
db.emp.find({Nome: "Ana"});
db.emp.update({Nome: "Ana"}, {$set: { slaario: 15000}});
db.emp.find({Nome: "Ana"});
```

Código-fonte 45 – Exemplo de uso do operador `$set`
Fonte: Elaborado pelo autor (2022)

- `$rename` renomeia um campo.

```
db.emp.find({Nome: "Ana"});
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     db.emp.update({Nome:"Ana"},{$rename:{"slaario":"salario"
}});
     db.emp.find({Nome: "Ana"});
```

Código-fonte 46 – Exemplo de uso do operador `$rename`
Fonte: Elaborado pelo autor (2022)

- `$unset` remove um campo específico de um documento.

```
     db.emp.find({Nome: "Ana"});
     db.emp.update({Nome:"Ana"},{$unset:{salario:""}});
     db.emp.find({Nome: "Ana"});
```

Código-fonte 47 – Exemplo de uso do operador `$rename`
Fonte: Elaborado pelo autor (2022)

O quadro abaixo compara o uso de comandos SQL dos bancos de dados relacional e o MongoDB.

Exemplo Em SQL	Exemplo em MongoDB
UPDATE emp SET Idade = Idade + 2 WHERE Nome = 'Ana'	db.emp.update({Nome: "Ana"}, {$inc: {Idade: 2}}, {multi: true})
UPDATE emp SET Idade = 33 WHERE Nome = 'Ana'	`db.emp.update({Nome: "Ana"}, {$set: {Idade: 33}}, {multi: true})`

Quadro 13 – Quadro de equivalência de operadores de atualização entre SQL versus MongoDB
Fonte: Adaptado de MongoDB (2022)

10 ELIMINANDO DADOS

O método `remove()` remove todos os documentos que atendam as condições de pesquisa especificadas. A opção `justOne` limita a operação de remoção a um único documento (MONGODB, 2022). Sua sintaxe é:

```
     db.coleção.remove(consulta,{justOne:<boolean>})
```

Código-fonte 48 – Sintaxe do método `remove()`
Fonte: MongoDB (2022)

Onde:

Coleção – Indica o nome da coleção.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Consulta – Indica quais serão os critérios para pesquisar o documento que será removido.

`justOne` – Indica que apenas um documento será removido.

Se todos os documentos forem eliminados, mesmo assim, a coleção continuará existindo. Caso queira eliminar a coleção, use o método `drop()`;

Exemplo de uso:

```
db.apaga.insert({cod:7369,nome:"Maria",cargo:"DBA"});
db.apaga.insert({cod:7499,nome:"Rosa",cargo:"DBA"});
db.apaga.insert({cod:7521,nome:"Ana",cargo:"DBA"});
db.apaga.insert({cod:7566,nome:"Rita",cargo:"DBA"});
db.apaga.find();
db.apaga.remove({cod:{$gt:7521}});
db.apaga.find();
db.apaga.remove({cargo:"DBA"},true);
db.apaga.find();
db.apaga.remove({});
db.apaga.find();
show collections;
db.apaga.drop();
show collections;
```

Código-fonte 49 – Exemplo de remoção documentos de uma coleção
Fonte: Elaborado pelo autor (2022)

O quadro abaixo compara o uso de comandos SQL dos bancos de dados relacional e o MongoDB.

Exemplo Em SQL	Exemplo em MongoDB
DELETE FROM apaga WHERE nome = ‘Ana’	`db.users.remove({name: "Ana"})`

Quadro 14 – Quadro de equivalência de operações de remoção entre SQL versus MongoDB
Fonte: Adaptado de MongoDB (2022)

11 INSTALAÇÃO MONGODB

O MongoDB pode ser instalado em diversos sistemas operacionais e possui várias versões diferentes. Esta instalação cobre a instalação do MongoDB Community Edition no sistema operacional Windows usando o instalador padrão do software.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

11.1 Download do instalador

Faça o download do instalador do MongoDB Community Edition .msi através do MongoDB Download Center (https://www.mongodb.com/try/download/community).

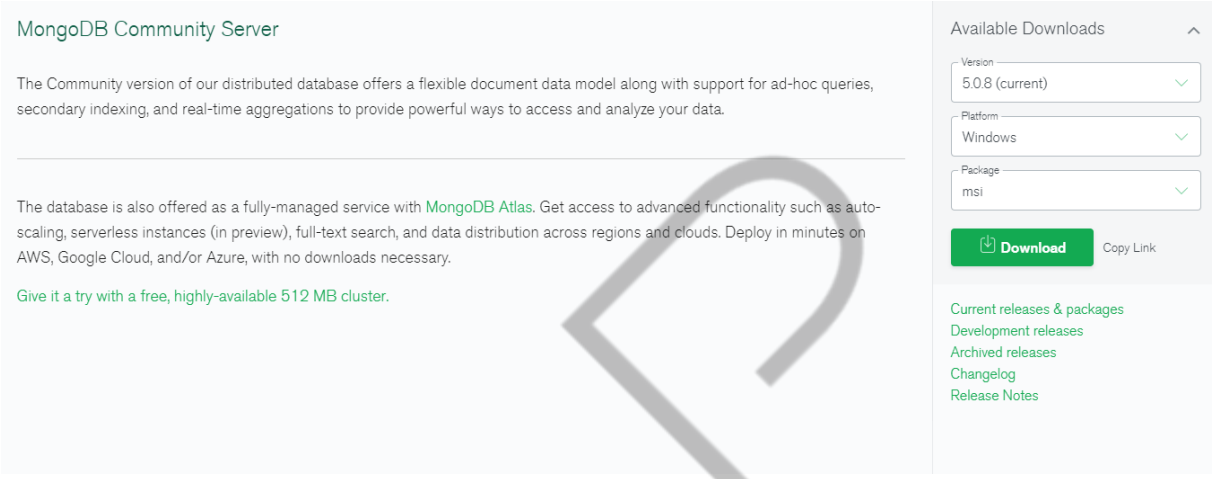


Figura 13 – MongoDB Download Center
Fonte: MongoDB (2022)

- Em Version (versão), selecione a versão do MongoDB para download.
- Em Plataform (Plataforma), selecione Windows.
- Em Na lista suspensa Pacote, selecione msi.
- Clique em Download (baixar).

11.2 Execute o instalador do MongoDB Community Edition

Após o término do download do instalador do MongDB Community Edition:

- Vá à pasta onde fez o download do instalador do MongoDB Community Editon. Nas configurações padrão do Windows, os downloads de arquivos são feitos na pasta Downloads. O instalador é um arquivo com a extensão .msi.
- Dê um duplo clique no arquivo com a extensão .msi.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

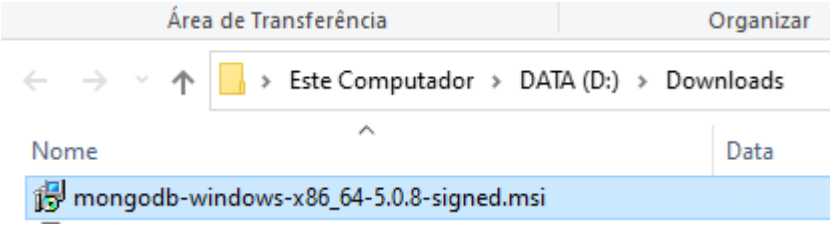


Figura 14 – Instalador do MongoDB Community Edition
Fonte: MongoDB (2022)

11.3 Siga os passos do instalador do MongoDB Community Edition

Clique em Next.

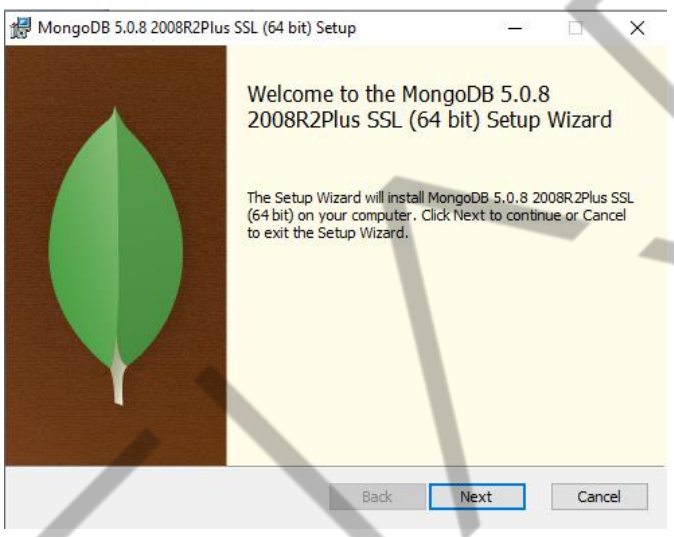
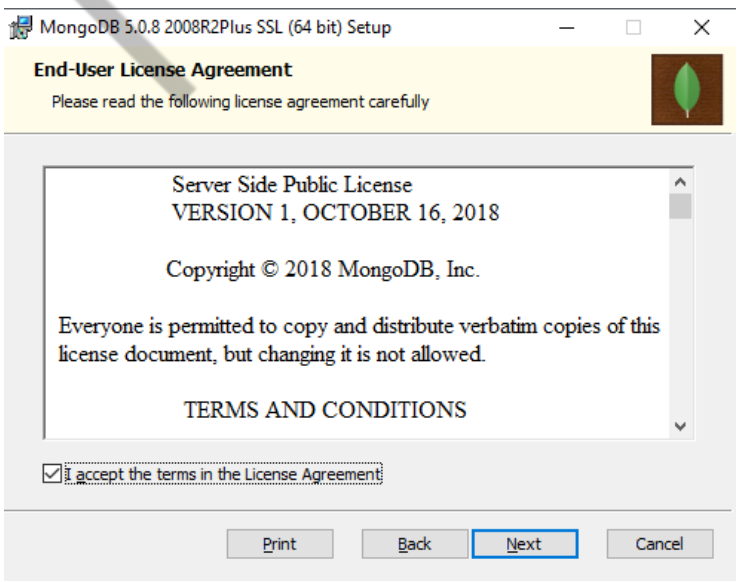


Figura 15 – Primeira tela do Instalador do MongoDB Community Edition
Fonte: MongoDB (2022)

Marque a caixa aceitando os termos e condições e clique em Next.



PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 16 – Termos e condições do Instalador do MongoDB Community Edition
Fonte: MongoDB (2022)

Clique em Complete e clique em Next.

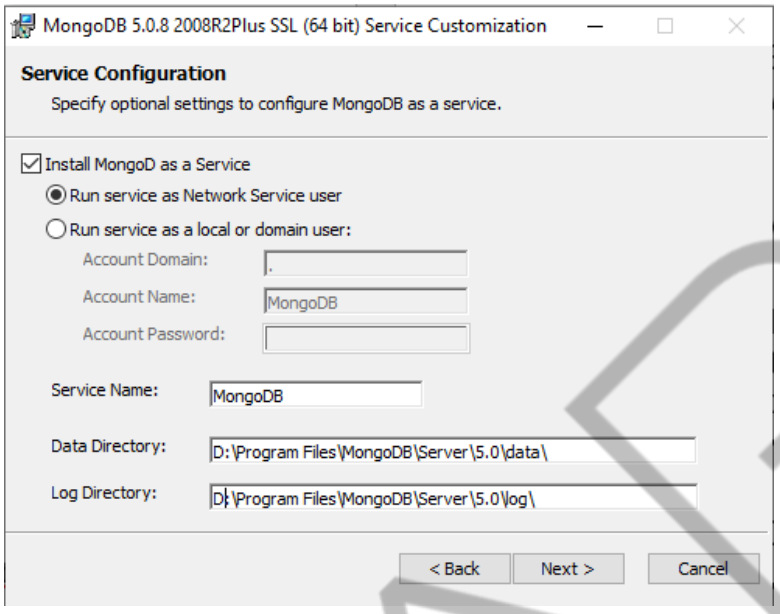


Figura 17 – Configurações do Instalador do MongoDB Community Edition
Fonte: MongoDB (2022)

Clique em Install.

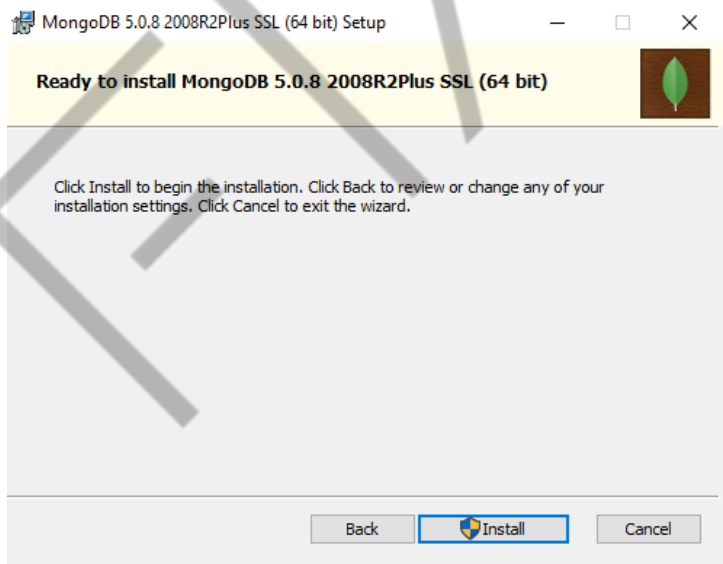


Figura 18 – Passos finais do Instalador do MongoDB Community Edition
Fonte: MongoDB (2022)

Aguarde a instalação completar e clique em Finish.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

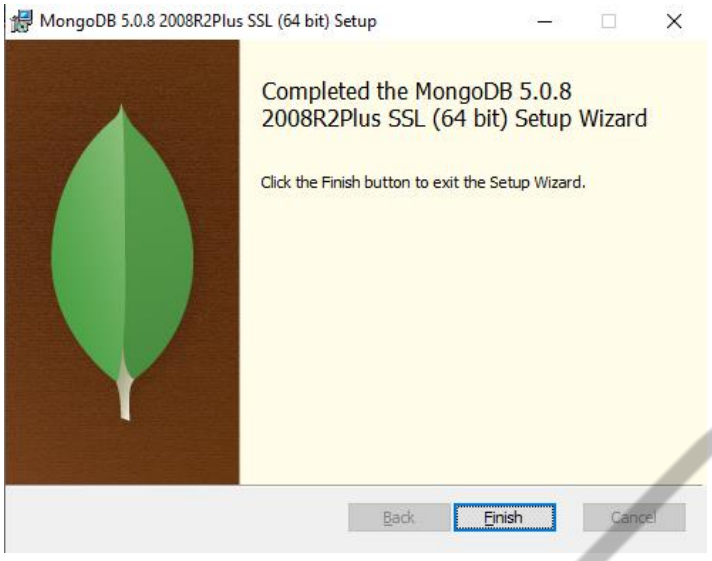


Figura 19 – Tela final do Instalador do MongoDB Community Edition
Fonte: MongoDB (2022)

Pronto! A instalação do MongoDB está finalizada.

14 ESTUDO DE CASO

A Cisco é líder mundial em redes, transformando a forma como as pessoas se conectam, se comunicam e colaboram. Em novembro de 2011, a Cisco lançou o WebEx Social (anteriormente conhecido como Quad), uma plataforma de colaboração empresarial projetada para a força de trabalho social, móvel, visual e virtual de hoje. Um “Facebook para empresas”, o WebEx Social conecta as pessoas às informações e conhecimentos de que precisam, quando precisam. Os usuários compartilham conhecimentos e ideias em toda a empresa e além das fronteiras geográficas e organizacionais. Quando seu banco de dados relacional existente precisava de uma extensão NoSQL para alimentar o espaço de trabalho colaborativo, a Cisco optou pelo MongoDB como seu banco de dados NoSQL preferido.

14.1 O problema

O WebEx Social é uma verdadeira plataforma Enterprise 2.0, “um lugar onde as pessoas 'vivem' no trabalho”, de acordo com Biren Gandhi, arquiteto principal do Cloud Collaboration Applications Technology Group (CCATG) da Cisco. A plataforma de colaboração abrangente combina os quatro pilares principais do Enterprise 2.0 –

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

processos de negócios, conteúdo, comunicação e recursos sociais – permitindo que os usuários trabalhem juntos sem esforço. Com seu banco de dados relacional existente, consultas SQL complexas em esquemas altamente normalizados consumiam tempo e a Cisco tinha pouco espaço para escalar horizontalmente. Além disso, era difícil gerenciar atualizações de esquema e migrar dados de versão para versão.

A rápida evolução do WebEx Social exigiu recursos expandidos de armazenamento de dados e a capacidade de criar recursos sociais bem-sucedidos. A Cisco buscou uma solução NoSQL de código aberto que fosse escalável, flexível e atendesse às necessidades corporativas do produto: suporte comercial seguro e maduro, capaz de fornecer backup e recuperação de desastres consistentes e fácil de integrar com sistemas legados. Após uma extensa avaliação e uma prova de conceito bem-sucedida, a Cisco adotou o MongoDB em meados de 2011.

14.2 Por que MongoDB?

O MongoDB agora serve como o principal armazenamento de dados em tempo real para os recursos sociais do WebEx Social que são de natureza pesada para gravação. Por exemplo, um feed de atividade social que notifica milhares de usuários dependendo de algumas regras complexas de privacidade e visibilidade. Além disso, a Cisco usa o MongoDB para realizar algumas análises de redes sociais. Com o recurso MapReduce do MongoDB, os desenvolvedores da Cisco podem gerar automaticamente, com facilidade, recomendações como usuários para se conectar, comunidades para participar e conteúdo interessante para visualizar, bem como estatísticas, incluindo principais colaboradores, colegas mais populares e documentos mais baixados.

"O MongoDB é um projeto de código aberto muito ativo com uma comunidade vibrante. Ele combina com o conjunto de recursos do WebEx Social e se ajusta exatamente às nossas necessidades", disse Gandhi.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

14.3 Modelo de documento fácil de implementar

O modelo de dados baseado em documentos do MongoDB oferece à Cisco o equilíbrio certo entre bancos de dados relacionais complexos e o armazenamento de valor de chave puro da maioria das soluções NoSQL.

O mapeamento intuitivo do modelo de domínio para documentos JSON e recursos de consulta complexos permitem que a Cisco faça referência cruzada de usuários e consulte listas incorporadas, funções que não eram possíveis em seu banco de dados existente ou eram caras de executar. Em contraste com um modelo relacional, que requer um processo de três etapas de fetch-update-commit, as atualizações atômicas no local do MongoDB permitem “operações one-stop”, de acordo com Gandhi.

14.4 Tempo de resposta de milissegundos

Com o MongoDB, a Cisco acelerou as leituras de 30 segundos em alguns casos extremos para dezenas de milissegundos por objeto e eliminou a necessidade de cache em certos casos. À medida que a plataforma WebEx Social evolui, a Cisco planeja migrar mais recursos de gravação pesada para o MongoDB.

14.5 Altamente escalável

O MongoDB atende às demandas da atividade de gravação de alto volume do WebEx Social. O MongoDB oferece uma maneira rápida e fácil de desnormalizar, indexar e extrair informações de armazenamentos de dados e seu composto exclusivo e indexação geoespacial são uma grande vantagem.

14.6 Alta disponibilidade

A Cisco usa conjuntos integrados de sharding e réplicas, que são fáceis de usar e permitem alta disponibilidade por meio de failover automático de nós e recuperação de nós membros.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Como parte de várias iniciativas de inovação na Cisco, muitos desenvolvedores participam de projetos voluntários em que as pessoas podem colaborar e compartilhar os resultados de suas ideias. Muitos adotaram o MongoDB como parte do desenvolvimento de seus projetos, uma prova da usabilidade, flexibilidade e inovação que o banco de dados oferece.

14.7 Resultados

A equipe de desenvolvimento da Cisco está constantemente avaliando o roteiro de produto mais eficaz para impulsionar o sucesso do cliente no WebEx Social. O MongoDB fornece à Cisco uma plataforma amigável para clientes e uma tecnologia fácil e poderosa para desenvolvedores. A Cisco está focada em lançar novos recursos rapidamente e oferecer flexibilidade aos usuários, e “o MongoDB é uma grande parte disso”, disse Gandhi.

REFERÊNCIAS

BUCKLER, Craig. **SQL vs NoSQL: The Differences**. 2015. Disponível em: <https://www.sitepoint.com/sql-vs-nosql-differences/>. Acesso em: 30 jun. 2022.

EVANS, Eric. **NoSQL: What's in a name?**. 2009. Disponível em: < http://blog.sym-link.com/posts/2009/30/nosql_whats_in_a_name/>. Acesso em: 26 maio 2022.

GEORGIEV, Georgi. **What is Vertical scaling and Horizontal scaling – Vertical and Horizontal hardware / services scaling**. 2014. Disponível em: <http://www.pc-freak.net/blog/vertical-horizontal-server-services-scaling-vertical-horizontal-hardware-scaling/>. Acesso em: 01 jul. 2022.

HOWS, David; MEMBREY, Peter; PLUGGE, Eelco. **Introdução ao MongoDB.** Brasil: Novatec, 2015.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

MENG, Alex. **Cassandra Query Language CQL vs SQL**. 2014. Disponível em: <https://medium.com/@alexbmeng/cassandra-query-language-cql-vs-sql-7f6ed7706b4c>. Acesso em: 30 jun. 2022.

MONGODB, Corporate. **The MongoDB 4.0 Manual**. 2022. Disponível em: <https://docs.mongodb.com/manual/>. Acesso em: 01 jul. 2022.

PUGA, Sandra; França, Edson; Goya, Milton. **Banco de Dados: Implementação PL/SQL e Oracle 11g**. São Paulo: Pearson, 2013.

SADALAGE, Pramod; FOWLER, Martin. **NoSQL Essencial.** Brasil: Novatec, 2013.

STROZZI, Carlo. **NoSQL Strozzi**. Disponível em: <http://www.strozzi.it/ cgi-bin/CSA/tw7/I/en_US/nosql/Home Page>. Acesso em: 29 jun. 2022.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com